

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

**Лабораторная работа №5
По предмету: “Вычислительная математика”
Вариант № 11**

Выполнил: Федоров Д. А.

Р3206

Проверил: Зинченко А. А.

г. Санкт-Петербург 2026 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ 3

1. 3

2. 3

3. 3

4. 4

Исходный код программы:..... 4

Результаты работы программы:..... 5

Заключение:..... 7

ЗАДАНИЕ:

Вычислительная реализация задачи:

1. Выбрать из табл. 1 заданную по варианту таблицу $y = f(x)$ (таблица 1.1 – таблица 1.5);
2. Построить таблицу конечных разностей для заданной таблицы. Таблицу отразить в отчете;
3. Вычислить значения функции для аргумента X_1 (см. табл.1), используя первую или вторую интерполяционную формулу Ньютона. Обратит внимание какой конкретно формулой необходимо воспользоваться;
4. Вычислить значения функции для аргумента X_2 (см. табл. 1), используя первую или вторую интерполяционную формулу Гаусса. Обратит внимание какой конкретно формулой необходимо воспользоваться;
5. Вычислить значения функции для аргументов X_1 и X_2 с помощью **многочлена** Лагранжа;
6. *Подробные вычисления привести в отчете.*

Программная реализация задачи:

1. Исходные данные задаются тремя способами:
 - a) в виде набора данных (таблицы x, y), пользователь вводит значения с клавиатуры;
 - b) в виде сформированных в файле данных (подготовить не менее трех тестовых вариантов);
 - c) на основе выбранной функции, из тех, которые предлагает программа, например, $\sin x$. Пользователь выбирает уравнение, исследуемый интервал и количество точек на интервале (не менее двух функций).
2. Сформировать и вывести таблицу разделенных/конечных разностей (в зависимости от варианта);
3. Вычислить приближенное значение функции для заданного значения аргумента, введенного с клавиатуры, указанными методами (см. табл. 2). Сравнить полученные значения;

-
4. Построить графики заданной функции с отмеченными узлами интерполяции и интерполяционного многочлена Ньютона/Гаусса (разными цветами);
 5. Программа должна быть протестирована на различных наборах данных, в том числе и некорректных.
 6. Проанализировать результаты работы программы.

Необязательное задание (до 15 баллов)

1. Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Стирлинга;
2. Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Бесселя.

Вычислительная реализация задачи:

№ варианта	X1	X2
11	0,255	0,405

x	y
0,25	1,2557
0,30	2,1764
0,35	3,1218
0,40	4,0482
0,45	5,9875
0,50	6,9195
0,55	7,8359

Таблица конечных разностей:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
0,25	1,2557	0,9207	0,0247	-0,0437	1,0756	-4,1277	10,1917
0,30	2,1764	0,9454	-0,0190	1,0319	-3,0521	6,0640	
0,35	3,1218	0,9264	1,0129	-2,0202	3,0119		
0,40	4,0482	1,9393	-1,0073	0,9917			
0,45	5,9875	0,9320	-0,0156				
0,50	6,9195	0,9164					
0,55	7,8359						

Конечные разности первого порядка:

$$\Delta y_i = y_{i+1} - y_i, i = 0, 1, \dots, n - 1$$

Конечные разности второго порядка:

$$\Delta^2 y_i = \Delta y_{i+1} - \Delta y_i$$

Конечные разности k-го порядка:

$$\Delta^k y_i = \Delta_{i+1}^{(k-1)y} - \Delta^{k-1} y_i$$

Значения функции для аргумента X_1 :

$$X_1 = 0,255, h = 0,05$$

$$t = \frac{x - x_0}{h} = \frac{0,255 - 0,25}{0,05} = 0,1$$

Первая интерполяционная формула Ньютона:

$$Nn(x) = y_0 + t \cdot \Delta y_0 + \frac{t(t-1)}{2!} \cdot \Delta^2 y_0 + \frac{t(t-1)(t-2)}{3!} \cdot \Delta^3 y_0 + \dots + \frac{t(t-1)\dots(t-n+1)}{n!} \cdot \Delta^n y_0 = 1,2567$$

Значения функции для аргумента X_2 :

$$X_2 = 0,405, h = 0,05$$

$$t = \frac{x - a}{h} = \frac{0,405 - 0,4}{0,05} = 0,1$$

Центральная точка $a = 0,40$; $X_2 > a \Rightarrow$ берём первую интерполяционную формулу Гаусса:

$$\begin{aligned} Pn(x) = & y_0 + t \cdot \Delta y_0 + \frac{t(t-1)}{2!} \cdot \Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!} \cdot \Delta^3 y_{-1} \\ & + \frac{(t+1)t(t-1)(t-2)}{4!} \cdot \Delta^4 y_{-2} + \frac{(t+2)(t+1)t(t-1)(t-2)}{5!} \cdot \Delta^5 y_{-2} + \dots \\ & + \frac{(t+n-1)\dots(t-n+1)}{(2n-1)!} \cdot \Delta^{2n-1} y_{-(n-1)} \\ & + \frac{(t+n-1)\dots(t-n)}{(2n)!} \cdot \Delta^{2n} y_{-n} = 4,2174 \end{aligned}$$

Значения функции для аргументов X_1 и X_2 с помощью многочлена Лагранжа:

$$L_n(x) = \sum_{i=0}^n y_i \cdot \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

$$L_i(x) = \frac{(x - x_0)(x - x_1)\dots(x - x_{i-1})(x - x_{i+1})\dots(x - x_6)}{(x_i - x_0)(x_i - x_1)\dots(x_i - x_{i-1})(x_i - x_{i+1})\dots(x_i - x_6)}$$

Значение функции для X_1 :

$$L_0 = 0,7766$$

$$L_1 = 0,5177$$

$$L_2 = -0,6131$$

$$L_3 = 0,5356$$

$$L_4 = -0,2986$$

$$L_5 = 0,0951$$

$$L_6 = -0,0132$$

$$L_n(X_1) = 1,2557 * 0,7766 + 2,1764 * 0,5177 + 3,1218 * (-0,6131) + 4,0482 * 0,5356 + 5,9875 * (-0,2986) + 6,9195 * 0,0951 + 7,8359 * (-0,0132) = 1,1228$$

Значение функции для X2:

$$L_0 = -0,0016$$

$$L_1 = 0,0140$$

$$L_2 = -0,0670$$

$$L_3 = 0,9820$$

$$L_4 = 0,0818$$

$$L_5 = -0,0155$$

$$L_6 = 0,0017$$

$$L_n(X_2) = 1,2557 * (-0,0016) + 2,1764 * 0,0140 + 3,1218 * (-0,0670) + 4,0482 * 0,9820 + 5,9875 * 0,0818 + 6,9195 * (-0,0155) + 7,8359 * 0,0017 = 4,1904$$

$$L_n(X_1) = 1,1228$$

$$L_n(X_2) = 4,1904$$

Программная реализация задачи:

Методы для реализации в программе:

- 1 - Многочлен Лагранжа,
- 2 - Многочлен Ньютона с разделенными разностями (1, 2 формулы),
- 4 - Многочлен Гаусса (1, 2 формулы).

Листинг программы:

```
import math

import numpy as np

import matplotlib.pyplot as plt

functions = {
    1: ("sin(x)", np.sin),
```

```
2: ("x^2", lambda x: x**2)
}
```

```
def read_from_file(filename="data.txt"):
```

```
    try:
```

```
        x, y = [], []
```

```
        with open(filename, 'r', encoding='utf-8') as f:
```

```
            for line in f:
```

```
                line = line.strip().replace(',', ' ')
```

```
                if not line:
```

```
                    continue
```

```
                parts = line.split()
```

```
                if len(parts) == 2:
```

```
                    x.append(float(parts[0]))
```

```
                    y.append(float(parts[1]))
```

```
        if not x:
```

```
            print("Ошибка: файл не содержит корректных данных.")
```

```
            return None, None
```

```
        return np.array(x), np.array(y)
```

```
    except Exception as e:
```

```
        print(f"Ошибка при чтении файла: {e}")
```

```
        return None, None
```

```
def read_from_console():
```

```
    print("Введите x и y (каждая пара на новой строке через пробел, пустая строка -  
конец):")
```

```
    x, y = [], []
```

```
while True:
```

```
    line = input().strip().replace(", ", " ")
```

```
    if not line:
```

```
        if len(x) >= 2:
```

```
            break
```

```
        else:
```

```
            print(f"Нужно минимум 2 точки, введено {len(x)}.")
```

```
            continue
```

```
parts = line.split()
```

```
if len(parts) != 2:
```

```
    print("нужно ввести два числа через пробел.")
```

```
    continue
```

```
try:
```

```
    x.append(float(parts[0]))
```

```
    y.append(float(parts[1]))
```

```
except ValueError:
```

```
    print("некорректный ввод чисел.")
```

```
return np.array(x), np.array(y)
```

```
def read_from_function():
```

```
    print("\nДоступные функции:")
```

```
    for key, (name, _) in functions.items():
```

```
        print(f"{key}. {name}")
```

```
while True:
```

```
    try:
```



```

choice = int(input("\nВыберите номер функции: "))

if choice in functions:
    name, func = functions[choice]

    break

print("Ошибка: выберите корректный номер из списка.")
except ValueError:
    print("Ошибка: введите целое число.")

while True:
    try:
        line = input("Введите границы интервала [a, b] через пробел: ").replace(',', '.').split()

        if len(line) != 2:
            print("Ошибка: введите два числа.")
            continue

        a, b = float(line[0]), float(line[1])

        if a == b:
            print("Ошибка: границы не могут совпадать.")
            continue

        start, end = min(a, b), max(a, b)

        break
    except ValueError:
        print("Ошибка: введите корректные числа для границ.")

while True:
    try:
        n = int(input("Введите количество узлов (минимум 2): "))

        if n >= 2:
            break

        print("Ошибка: нужно минимум 2 узла")

```

```
except ValueError:
```

```
    print("Ошибка: введите целое число.")
```

```
x = np.linspace(start, end, n)
```

```
y = func(x)
```

```
print(f"{'x':<12}{'y':<12}")
```

```
for i in range(len(x)):
```

```
    print(f"{'x[i]:<12.4f'}{'y[i]:<12.4f}'")
```

```
return x, y
```

```
def get_finite_diff_table(y):
```

```
    n = len(y)
```

```
    table = np.zeros((n, n))
```

```
    table[:, 0] = y
```

```
    for j in range(1, n):
```

```
        for i in range(n - j):
```

```
            table[i, j] = table[i + 1, j - 1] - table[i, j - 1]
```

```
    return table
```

```
def get_divided_diff_table(x, y):
```

```
    n = len(y)
```

```
    table = np.zeros((n, n))
```

```
    table[:, 0] = y
```

```
    for j in range(1, n):
```

```
        for i in range(n - j):
```

```
            numerator = table[i + 1, j - 1] - table[i, j - 1]
```

```
            denominator = x[i + j] - x[i]
```

```
    table[i, j] = numerator / denominator

return table
```

```
def print_diff_table(x_nodes, table, title="конечных"):

    print(f"\nТаблица {title} разностей:")

    prefix = "f" if title == "разделенных" else "Δ^"

    header = f"{'xi':<12}{'yi':<12}"

    for i in range(1, table.shape[1]):

        col_name = f"{prefix}{i}" if title == "разделенных" else f"{prefix}{i}y"

        header += f"{col_name:<12}"

    print(header)

    for i in range(len(x_nodes)):

        row_str = f"{x_nodes[i]:<12.4f}{table[i, 0]:<12.4f}"

        for j in range(1, len(x_nodes) - i):

            row_str += f"{table[i, j]:<12.4f}"

        print(row_str)

def lagrange_interpolation(x_nodes, y_nodes, x_val):

    n = len(x_nodes)

    result = 0

    for i in range(n):

        polynomial = 1.0

        for j in range(n):

            if i != j:

                polynomial *= (x_val - x_nodes[j]) / (x_nodes[i] - x_nodes[j])

        result += y_nodes[i] * polynomial

    return result
```

```
def newton_forward_divided(x_nodes, div_table, x_val):
```

```
    n = len(x_nodes)
```

```
    result = div_table[0, 0]
```

```
    product = 1.0
```

```
    for i in range(1, n):
```

```
        product *= (x_val - x_nodes[i - 1])
```

```
        result += div_table[0, i] * product
```

```
    return result
```

```
def newton_backward_divided(x_nodes, div_table, x_val):
```

```
    n = len(x_nodes)
```

```
    result = div_table[n - 1, 0]
```

```
    product = 1.0
```

```
    for i in range(1, n):
```

```
        product *= (x_val - x_nodes[n - i])
```

```
        result += div_table[n - i - 1, i] * product
```

```
    return result
```

```
def gauss_first_formula(x_nodes, fin_table, x_val):
```

```
    n = len(x_nodes)
```

```
    mid = n // 2
```

```
    h = x_nodes[1] - x_nodes[0]
```

```
    t = (x_val - x_nodes[mid]) / h
```

```
    result = fin_table[mid, 0]
```

```
    p_term = 1.0
```

```
    for i in range(1, n):
```

```
        if i % 2 == 1:
```

```
            p_term *= (t + (i // 2)) / i
```

```

else:

    p_term *= (t - (i // 2)) / i

    row = mid - (i // 2)

    if 0 <= row < n - i:

        result += p_term * fin_table[row, i]

return result

```

```

def gauss_second_formula(x_nodes, fin_table, x_val):

```

```

    n = len(x_nodes)

    mid = n // 2

    h = x_nodes[1] - x_nodes[0]

    t = (x_val - x_nodes[mid]) / h

    result = fin_table[mid, 0]

    p_term = 1.0

    for i in range(1, n):

        if i % 2 == 1:

            p_term *= (t - (i // 2)) / i

        else:

            p_term *= (t + (i // 2)) / i

        row = mid - ((i + 1) // 2)

        if 0 <= row < n - i:

            result += p_term * fin_table[row, i]

    return result

```

```

def plot_results(x_nodes, y_nodes, target_x, div_table, fin_table):

```

```

    x_min, x_max = np.min(x_nodes), np.max(x_nodes)

    margin = (x_max - x_min) * 0.1

    x_plot = np.linspace(x_min - margin, x_max + margin, 200)

```

```

plt.figure(figsize=(12, 7))

plt.scatter(x_nodes, y_nodes, color='black', s=60, zorder=5, label='Узлы интерполяции')


y_lagrange = [lagrange_interpolation(x_nodes, y_nodes, xi) for xi in x_plot]

plt.plot(x_plot, y_lagrange, color='blue', linewidth=2, label='Многочлен Лагранжа')


y_newton = [newton_forward_divided(x_nodes, div_table, xi) for xi in x_plot]

plt.plot(x_plot, y_newton, color='green', linestyle='--', linewidth=2, label='Многочлен
Ньютона')


is_equidistant = np.allclose(np.diff(x_nodes), x_nodes[1] - x_nodes[0])

if is_equidistant and len(x_nodes) > 1:

    y_gauss = [gauss_first_formula(x_nodes, fin_table, xi) for xi in x_plot]

    plt.plot(x_plot, y_gauss, color='magenta', linestyle=':', linewidth=2, label='Многочлен
Гаусса (1-я ф.)')


plt.axvline(target_x, color='red', alpha=0.3, label=f'Искомый x = {target_x}')


plt.xlabel('x')
plt.ylabel('y')

plt.title('Сравнение интерполяционных многочленов')

plt.legend(loc='best')

plt.grid(True, alpha=0.3)

plt.tight_layout()

plt.show()

```

```

def stirling_interpolation(x_nodes, fin_table, x_val):

    n, mid = len(x_nodes), len(x_nodes) // 2

    h = x_nodes[1] - x_nodes[0]

    t = (x_val - x_nodes[mid]) / h

    result = fin_table[mid, 0]

    for i in range(1, (n // 2) + 1):

        term1 = 1.0

        for k in range(i - 1):

            term1 *= (t**2 - k**2)

        fact_odd = math.factorial(2 * i - 1)

        if mid - i >= 0 and mid - i + 1 < n:

            diff_avg = (fin_table[mid - i, 2 * i - 1] + fin_table[mid - i + 1, 2 * i - 1]) / 2

            result += (t * term1 / fact_odd) * diff_avg

        term2 = term1 * (t**2 - (i - 1)**2) if i > 1 else t**2

        fact_even = math.factorial(2 * i)

        if mid - i >= 0:

            result += (term2 / fact_even) * fin_table[mid - i, 2 * i]

    return result

```

```

def besel_interpolation(x_nodes, fin_table, x_val):

    n = len(x_nodes)

    mid = n // 2

    h = x_nodes[1] - x_nodes[0]

    t = (x_val - x_nodes[mid]) / h

```

```
result = (fin_table[mid, 0] + fin_table[mid + 1, 0]) / 2
```

```
result += (t - 0.5) * fin_table[mid, 1]
```

```
for i in range(1, n // 2 + 1):
```

```
    factor_even = 1.0
```

```
    factor_even *= t * (t - 1)
```

```
    for k in range(1, i):
```

```
        factor_even *= (t + k) * (t - k - 1)
```

```
    fact_even = 1
```

```
    for j in range(2, 2 * i + 1):
```

```
        fact_even *= j
```

```
    if mid - i >= 0 and mid - i + 1 < n and 2 * i < n:
```

```
        avg_diff = (fin_table[mid - i, 2 * i] + fin_table[mid - i + 1, 2 * i]) / 2
```

```
        result += (factor_even / fact_even) * avg_diff
```

```
    factor_odd = (t - 0.5) * t * (t - 1)
```

```
    for k in range(1, i):
```

```
        factor_odd *= (t + k) * (t - k - 1)
```

```
    fact_odd = 1
```

```
    for j in range(2, 2 * i + 2):
```

```
        fact_odd *= j
```

```
    if mid - i >= 0 and 2 * i + 1 < n:
```

```
        result += (factor_odd / fact_odd) * fin_table[mid - i, 2 * i + 1]
```



```
return result
```

```
def main():
```

```
    print("1. Консоль, 2. Файл, 3. Функция")
```

```
    choice = input("")
```

```
    if choice == '1':
```

```
        x, y = read_from_console()
```

```
    elif choice == '2':
```

```
        x, y = read_from_file()
```

```
    else:
```

```
        x, y = read_from_function()
```

```
    if x is None:
```

```
        return
```

```
    target_x = float(input("\nВведите X: ").replace(',', '.'))
```

```
    finite_table = get_finite_diff_table(y)
```

```
    divided_table = get_divided_diff_table(x, y)
```

```
    print_diff_table(x, finite_table, "конечных")
```

```
    print_diff_table(x, divided_table, "разделенных")
```

```
    print(f"Лагранж: {lagrange_interpolation(x, y, target_x):.6f}")
```

```
    n = len(x)
```

```
    if target_x <= x[n // 2]:
```

```
        print(f"Ньютон (вперед): {newton_forward_divided(x, divided_table, target_x):.6f}")
```

```
    else:
```

```
        print(f"Ньютон (назад): {newton_backward_divided(x, divided_table, target_x):.6f}")
```

```
is_equidistant = np.allclose(np.diff(x), x[1] - x[0])
```

```
if is_equidistant:
```

```
    mid = n // 2
```

```
    if target_x >= x[mid]:
```

```
        print(f"Гаусс (1-я ф.): {gauss_first_formula(x, finite_table, target_x):.6f}")
```

```
    else:
```

```
        print(f"Гаусс (2-я ф.): {gauss_second_formula(x, finite_table, target_x):.6f}")
```

```
    print(f"Стирлинг: {stirling_interpolation(x, finite_table, target_x):.6f}")
```

```
    print(f"Бессель: {bessel_interpolation(x, finite_table, target_x):.6f}")
```

```
else:
```

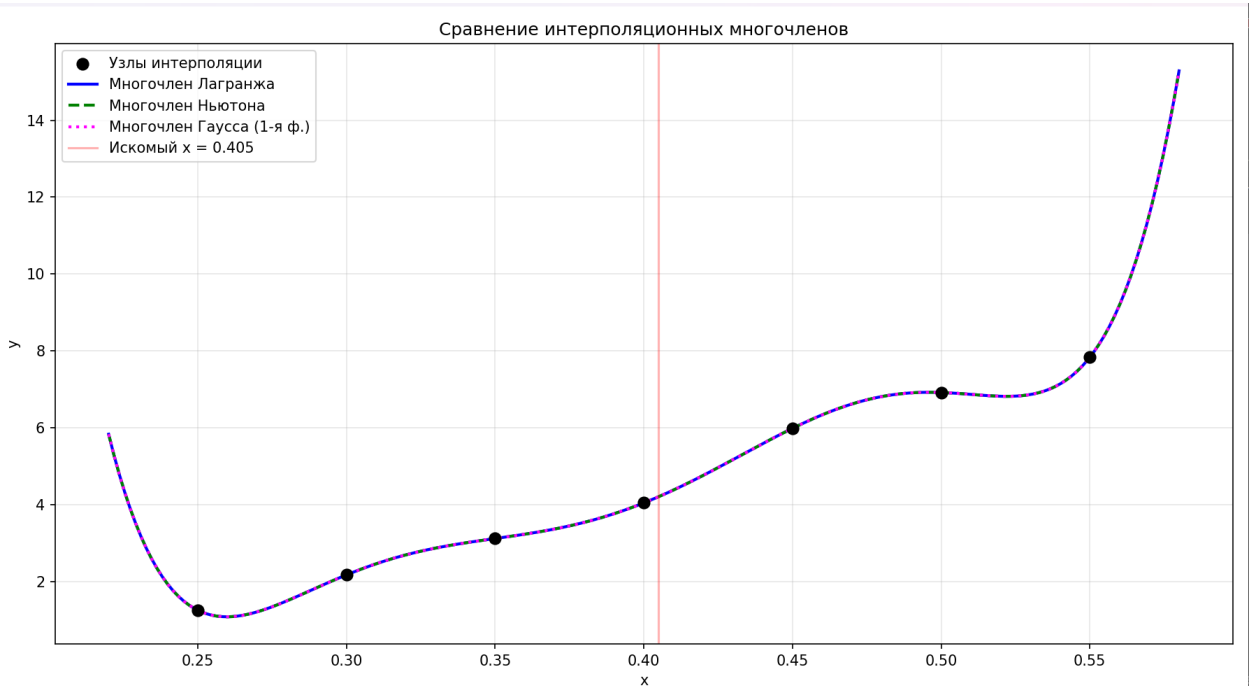
```
    print("Центральные методы недоступны (нужен равный шаг)")
```

```
plot_results(x, y, target_x, divided_table, finite_table)
```

```
if __name__ == "__main__":
```

```
    main()
```

Результаты работы программы:



```
PS D:\vichmat> & C:/Users/HP/AppData/Local/Programs/Python/Python311/python.exe d:/vichmat/lab5.py
1. Консоль, 2. Файл, 3. Функция
1
Введите x и y (каждая пара на новой строке через пробел, пустая строка - конец):
0,25 1,2557
0,30 2,1764
0,35 3,1218
0,40 4,0482
0,45 5,9875
0,50 6,9195
0,55 7,8359

Введите X: 0,405

Таблица конечных разностей:
xi yi Δ^1y Δ^2y Δ^3y Δ^4y Δ^5y Δ^6y
0.2500 1.2557 0.9207 0.0247 -0.0437 1.0756 -4.1277 10.1917
0.3000 2.1764 0.9454 -0.0190 1.0319 -3.0521 6.0640
0.3500 3.1218 0.9264 1.0129 -2.0202 3.0119
0.4000 4.0482 1.9393 -1.0073 0.9917
0.4500 5.9875 0.9320 -0.0156
0.5000 6.9195 0.9164
0.5500 7.8359

Таблица разделенных разностей:
xi yi f1 f2 f3 f4 f5 f6
0.2500 1.2557 18.4140 4.9400 -58.2667 7170.6667 -110072.0000005928.8889
0.3000 2.1764 18.9080 -3.8000 1375.8667 -20347.3333 161706.6667
0.3500 3.1218 18.5280 202.5000 -2693.6000 20079.3333
0.4000 4.0482 38.7860 -201.4600 1322.2667
0.4500 5.9875 18.6400 -3.1200
0.5000 6.9195 18.3280
0.5500 7.8359

Лагранж: 4.209708
Ньютон (назад): 4.209708
Гаусс (1-я ф.): 4.209708
Стирлинг: 4.198277
Бессель: 4.217816
```

Вывод: В данной лабораторной работе я изучил методы интерполяции функций.